



ALTEN

كلية العلوم والتقنيات فاس
+٠٢٤٧.١+ | +٢.٠٠.١٤١ ٨ +٠١٤٨٤+٤١ - ٣.٠
FACULTÉ DES SCIENCES ET TECHNIQUES DE FÈS



Ingénierie Électrique et Systèmes Embarqués

Compte Rendu de :

Bases de Données

Encadré par *Prof. Loukili Yassine*

Mohammed AMANSOUR

Master IESE-1

mohammed.amansour@usmba.ac.ma

Introduction

Une **base de données** est un outil essentiel permettant d'organiser et de stocker des informations de manière structurée. On peut la comparer à un ensemble de tableaux (similaires à des fichiers Excel) où chaque fichier représente une **table**, chaque ligne un **enregistrement**, et chaque colonne un **attribut**.

L'objectif de ce rapport est de documenter le passage d'un besoin métier à une implémentation technique en suivant la méthode **MERISE**, qui est la méthode d'analyse la plus utilisée pour la conception de systèmes d'information.

Table des Matières

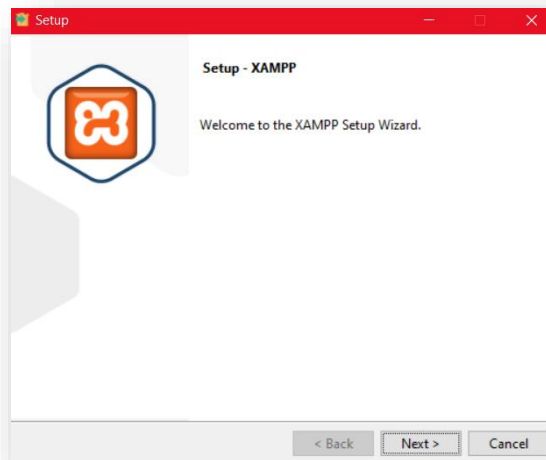
I. Configuration de l'Environnement : XAMPP.....	4
1. Processus d'Installation	4
2. Activation des Services	4
3. Accès à l'Interface de Gestion.....	5
Implémentation de la base STOCK	6
Introduction	6
Création et Configuration de l'Environnement	6
1. Création de la base de données	6
2. Création des tables et contraintes.....	6
3. Insertion des données	7
4. Analyse des stocks et des clients	8

I. Configuration de l'Environnement : XAMPP

Avant de passer au code, nous avons dû installer et configurer nos outils techniques. Nous avons choisi la distribution **XAMPP**.

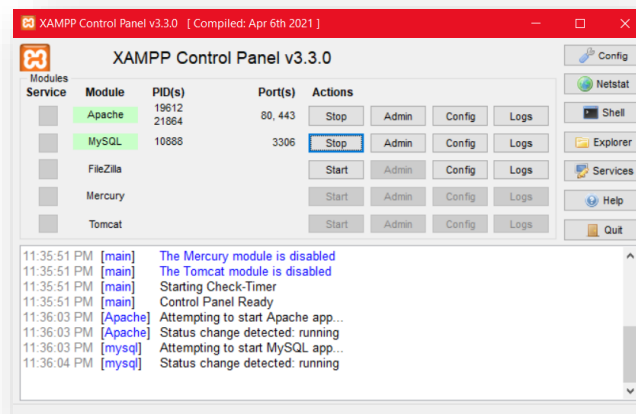
1. Processus d'Installation

Nous avons téléchargé l'installateur XAMPP et l'avons exécuté en veillant à installer Apache (le serveur web) et MySQL (notre système de gestion de base de données).



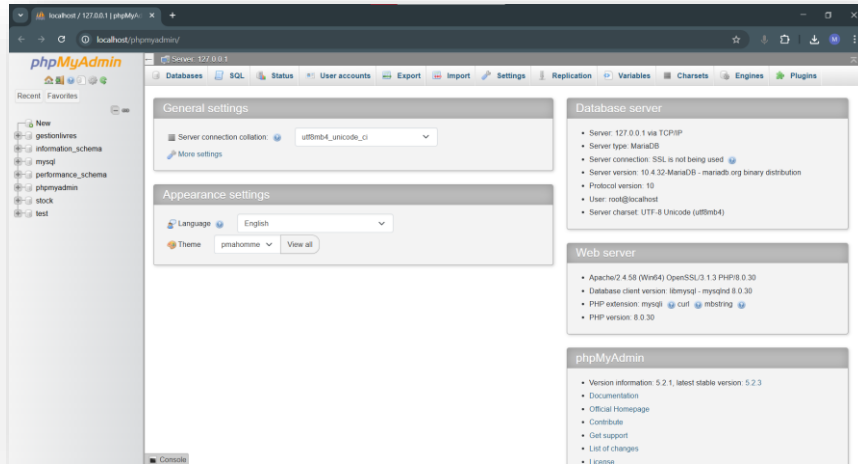
2. Activation des Services

Nous avons ouvert le panneau de contrôle XAMPP. Nous avons cliqué sur **"Start"** pour les modules Apache et MySQL. Une fois que les indicateurs sont passés au vert, notre serveur local était prêt.



3. Accès à l'Interface de Gestion

En cliquant sur le bouton "**Admin**" du module MySQL, nous avons accédé à l'interface **phpMyAdmin**, qui nous permet de saisir et d'exécuter nos requêtes SQL dans un navigateur.



Implémentation de la base STOCK

Introduction

Ce volet du rapport détaille l'implémentation physique de la base de données de gestion de stock nommée **STOCK**. L'objectif est de traduire le modèle relationnel en langage SQL (Structured Query Language) pour permettre la création, l'insertion et l'interrogation des données.

Création et Configuration de l'Environnement

1. Création de la base de données

Pour commencer, j'ai créé l'environnement de travail nécessaire pour héberger nos tables.

```
1. CREATE DATABASE STOCK;  
2. USE STOCK;
```

La commande *CREATE DATABASE* initialise l'espace de stockage nommé « STOCK ». L'instruction *USE* est cruciale pour diriger toutes les instructions suivantes vers cette base spécifique.

2. Création des tables et contraintes

J'ai conçu les tables en choisissant des types de données optimisés pour chaque attribut tout en définissant les clés primaires pour garantir l'unicité des enregistrements.

Table CLIENT

```
1. CREATE TABLE CLIENT (  
2.     nclt INT PRIMARY KEY,  
3.     nom VARCHAR(50) NOT NULL,  
4.     age INT,  
5.     adresse VARCHAR(100)  
6. );
```

- ***nclt*** : Identifiant unique du client.
- ***Nom/Adresse*** : Utilisation de *VARCHAR* pour économiser l'espace disque tout en permettant des chaînes de caractères de longueurs variables.

Table PROD

```
1. CREATE TABLE PROD (  
2.     nprod INT PRIMARY KEY,  
3.     design VARCHAR(50) NOT NULL,  
4.     couleur VARCHAR(20),  
5.     volume INT  
6. );
```

- ***nprod*** : Référence unique du produit (Clé Primaire).
- ***volume*** : Stocké en entier (INT) pour faciliter les tris et les calculs.

Table CMD

```
1. CREATE TABLE CMD (  
2.     nclt INT,  
3.     nprod INT,  
4.     qte INT NOT NULL,  
5.     date DATE,  
6.     PRIMARY KEY (nclt, nprod),  
7.     FOREIGN KEY (nclt) REFERENCES CLIENT(nclt),  
8.     FOREIGN KEY (nprod) REFERENCES PROD(nprod)  
9. );
```

- ***Clé Primaire Composite*** : J'ai combiné *nclt*, *nprod* et *date* comme clé primaire pour permettre à un même client de commander le même produit à des dates différentes.
- ***Clés Étrangères*** : Elles assurent que chaque commande est liée à un client et un produit existant.

3. Insertion des données

Pour cette étape, j'ai utilisé une méthode d'insertion groupée pour plus d'efficacité, en commençant par les clients, puis les produits et enfin les commandes pour respecter l'intégrité référentielle.

Insertion dans la table CLIENT

```
1. INSERT INTO CLIENT (nclt, nom, age, adresse) VALUES  
2. (10, 'Dupont', 45, '12 rue de la Paix'),  
3. (20, 'Martin', 22, '5 av. des Lilas'),  
4. (30, 'Dubois', 61, '30 bd. Voltaire'),  
5. (40, 'Leroy', 35, '8 Impasse du Sud'),  
6. (50, 'Robert', 55, '1 Place Royale');
```

Insertion dans la table PROD

```
1. INSERT INTO PROD (nprod, design, couleur, volume) VALUES
2. (1, 'Marteau', 'gris', 200),
3. (2, 'Pince coupante', 'vert', 50),
4. (3, 'Tournevis', 'bleu', 30),
5. (4, 'Pince universelle', 'rouge', 70),
6. (5, 'Scie', 'bleu', 150);
```

Insertion dans la table CMD

```
1. INSERT INTO CMD (nclt, nprod, qte, date) VALUES
2. (10, 1, 2, '2025-10-01'),
3. (10, 4, 1, '2025-10-01'),
4. (20, 3, 5, '2025-10-05'),
5. (30, 1, 1, '2025-10-10'),
6. (30, 2, 2, '2025-10-10'),
7. (40, 5, 10, '2025-10-15'),
8. (50, 2, 1, '2025-10-20'),
9. (50, 4, 3, '2025-10-20'),
10. (50, 5, 1, '2025-10-20');
```

4. Analyse des stocks et des clients

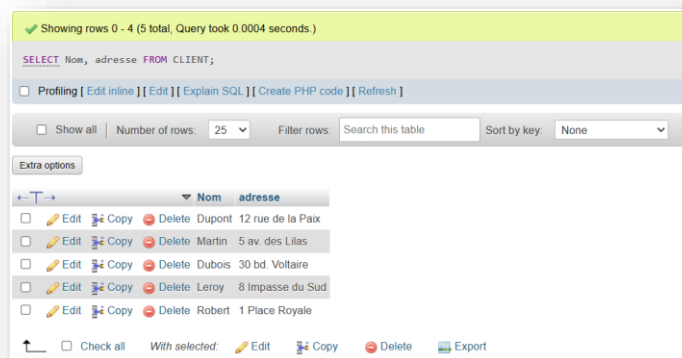
Dans cette dernière phase, j'ai formulé des requêtes SQL pour extraire des informations précises et répondre aux besoins de gestion du stock. Chaque requête a été testée pour garantir l'exactitude des résultats.

a. Liste complète des noms et adresses des clients

Premièrement, nous avons exécuté la commande :

```
1. SELECT Nom, adresse FROM CLIENT;
```

J'ai utilisé une projection simple pour ne récupérer que les colonnes pertinentes.

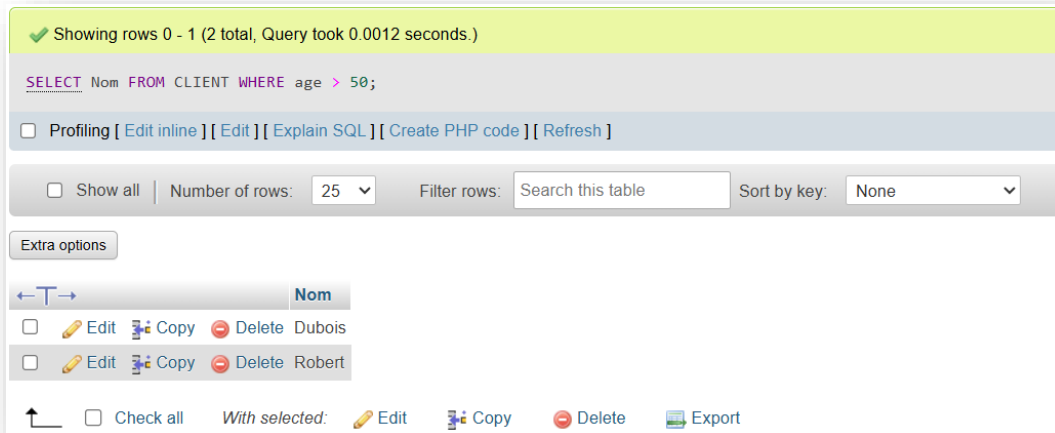


b. Clients âgés de plus de 50 ans

Nous avons exécuté la commande :

```
1. SELECT Nom FROM CLIENT WHERE age > 50;
```

Ici, j'applique une restriction *WHERE* sur l'attribut numérique age. La base de données filtre les lignes pour ne retourner que les clients concernés.

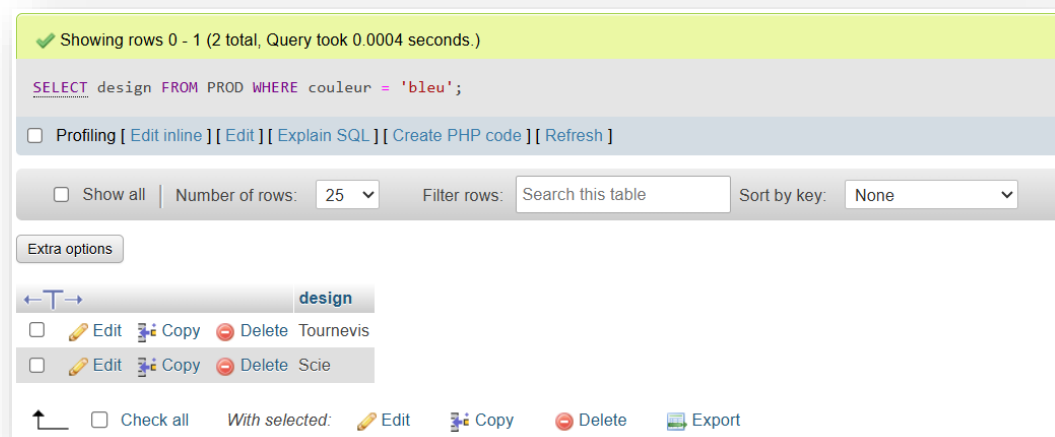


c. Design des produits de couleur bleue

Nous avons exécuté la commande :

```
1. SELECT design FROM PROD WHERE couleur = 'bleu';
```

Pour cette requête, j'ai fait attention à bien mettre la valeur 'bleu' entre apostrophes, car il s'agit d'une chaîne de caractères.



d. Produits verts OU volume supérieur à 100

Nous avons exécuté la commande :

```
1. SELECT * FROM PROD WHERE couleur = 'vert' OR volume > 100;
```

L'utilisation de l'opérateur logique *OR* est ici fondamentale. La requête renvoie les produits qui remplissent **au moins l'une** des deux conditions.

Showing rows 0 - 2 (3 total, Query took 0.0004 seconds.)

```
SELECT * FROM PROD WHERE couleur = 'vert' OR volume > 100;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	nprod	design	couleur	volume
☐ Edit ☐ Copy ☐ Delete	1	Marteau	gris	200
☐ Edit ☐ Copy ☐ Delete	2	Pince coupante	vert	50
☐ Edit ☐ Copy ☐ Delete	5	Scie	bleu	150

☐ Check all | With selected: ☐ Edit ☐ Copy ☐ Delete ☐ Export

e. Tri des clients par âge (du plus âgé au plus jeune)

Nous avons exécuté la commande :

```
1. SELECT * FROM CLIENT ORDER BY age DESC;
```

Pour organiser ma liste, j'ai utilisé *ORDER BY*. J'ai ajouté le mot-clé *DESC* pour que le client le plus âgé apparaisse en haut de la liste, conformément à la demande de tri décroissant.

Showing rows 0 - 4 (5 total, Query took 0.0004 seconds.) [age: 61... - 22...]

```
SELECT * FROM CLIENT ORDER BY age DESC;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	nclt	nom	age	adresse
☐ Edit ☐ Copy ☐ Delete	30	Dubois	61	30 bd. Voltaire
☐ Edit ☐ Copy ☐ Delete	50	Robert	55	1 Place Royale
☐ Edit ☐ Copy ☐ Delete	10	Dupont	45	12 rue de la Paix
☐ Edit ☐ Copy ☐ Delete	40	Leroy	35	8 Impasse du Sud
☐ Edit ☐ Copy ☐ Delete	20	Martin	22	5 av. des Lilas

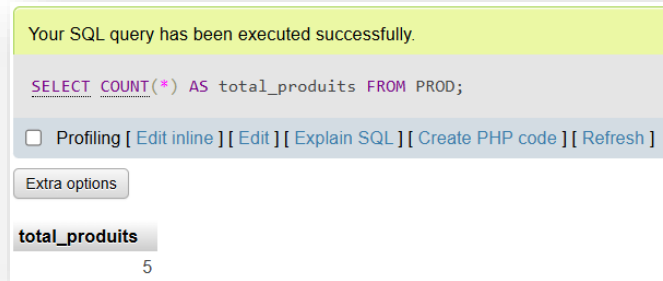
☐ Check all | With selected: ☐ Edit ☐ Copy ☐ Delete ☐ Export

f. Nombre total de produits en stock

Nous avons exécuté la commande :

```
1. SELECT COUNT(*) AS total_produits FROM PROD;
```

J'ai utilisé la fonction d'agrégation *COUNT* pour obtenir un résultat numérique unique représentant le volume global de références présentes dans la table *PROD*.

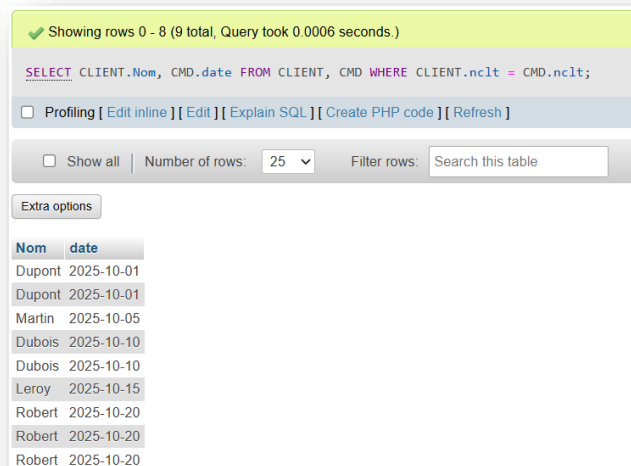


g. Noms des clients et dates de leurs commandes

Nous avons exécuté la commande :

```
1. SELECT CLIENT.Nom, CMD.date
2. FROM CLIENT, CMD
3. WHERE CLIENT.nclt = CMD.nclt;
```

Cette requête est cruciale car elle nécessite une **jointure**. En reliant la clé primaire *nclt* de la table *CLIENT* à la clé étrangère *nclt* de la table *CMD*, je peux associer un nom lisible à chaque transaction enregistrée.

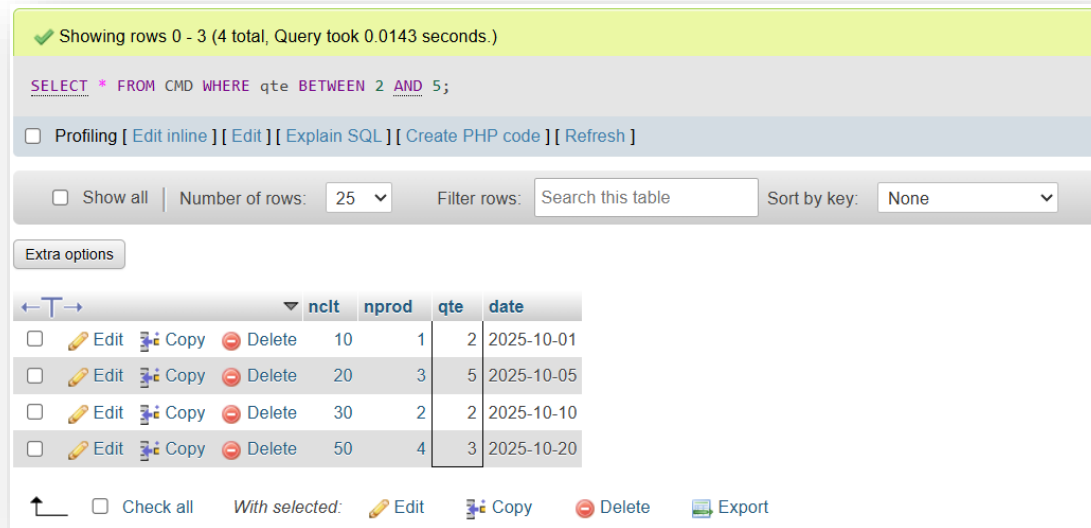


h. Commandes avec une quantité comprise entre 2 et 5

Nous avons exécuté la commande :

```
1. SELECT * FROM CMD WHERE qte BETWEEN 2 AND 5;
```

L'opérateur *BETWEEN* est le plus adapté ici car il inclut les bornes (2 et 5). C'est une syntaxe plus propre et plus lisible que d'écrire *qte >= 2 AND qte <= 5*.



Showing rows 0 - 3 (4 total, Query took 0.0143 seconds.)

`SELECT * FROM CMD WHERE qte BETWEEN 2 AND 5;`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				nclt	nprod	qte	date
☐	Edit	Copy	Delete	10	1	2	2025-10-01
☐	Edit	Copy	Delete	20	3	5	2025-10-05
☐	Edit	Copy	Delete	30	2	2	2025-10-10
☐	Edit	Copy	Delete	50	4	3	2025-10-20

☐ Check all | With selected: Edit Copy Delete Export

Conclusion

En conclusion, ce travail dirigé m'a permis de comprendre concrètement comment transformer un besoin de gestion en une solution technique fonctionnelle.

Ce TD confirme que la gestion rigoureuse des données est une compétence fondamentale pour un futur ingénieur, quel que soit son domaine de spécialisation.